

HelixPlay

HelixPlay

Превратите любую машину с GPU в собственное облачное игровое устройство.

Краткое описание

HelixPlay — это саморазвёртываемая облачная игровая платформа, которая превращает любую машину с поддержкой GPU в удалённый стриминговый хост, обеспечивая консольный уровень геймплея на настольных ПК, мобильных устройствах, телевизорах и в браузерах. Платформа построена как монорепозиторий на базе Go, состоящий из 46 субмодулей, с трёхслойным клиентом и возможностью белой маркировки для партнёров.

Краткая аннотация

HelixPlay — это саморазвёртываемая, открытая и поддерживающая белую маркировку облачная игровая платформа. Она превращает любую машину с GPU в удалённый стриминговый хост и обеспечивает консольный уровень геймплея на настольных ПК, мобильных устройствах, телевизорах и в браузерах через WebRTC/QUIC, с ядром на базе Go и клиентским стеком Wails/Flutter/Angular.

Подробное описание

HelixPlay — это облачная игровая платформа, построенная как монорепозиторий на базе Go, состоящий из 46 Git-субмодулей. Она превращает любой игровой ПК, который у вас уже есть, в стриминговый хост, обеспечивая консольный уровень геймплея на настольных ПК, мобильных устройствах, телевизорах и в браузерах. Платформа саморазвёртываемая,

открытая и поддерживает белую маркировку для партнёров. Суть предложения проста: ваше оборудование, ваш сервис, ваш бренд — никаких посредников в виде сторонних облаков.

Ключевое архитектурное решение — конвергенция трёхслойного клиента, ставка на жёсткую унификацию, которая окупается во всём остальном. Десктопное приложение на Wails, мобильное/TV-приложение на Flutter и веб-клиент на Angular работают поверх *единого* ядра Go, скомпилированного в WASM для браузера. Благодаря этому поведение пишется один раз и используется на всех платформах, а не дублируется в трёх вариантах. Под этим слоем реализован путь передачи медиаданных в реальном времени: захват → кодирование → пакетизация → передача → декодирование → рендеринг. Он интегрирован с платформенно-зависимыми средствами захвата (DXGI / ScreenCaptureKit / PipeWire) и аппаратными кодеками (NVENC / QSV / AMF / VideoToolbox), чтобы всю тяжёлую работу выполнял GPU. Передача данных осуществляется через WebRTC (Pion v4), QUIC (quic-go) и кастомные датаграммы UDP, выбранные с приоритетом на минимальную задержку, а не на удобство. Ядро бэкенда управляет сессиями, тенантами, каталогом и аутентификацией; агент хоста отвечает за захват, кодирование и передачу данных на периферии; а mDNS и механизм рандеву обеспечивают обнаружение хоста клиентами без ручной настройки.

HelixPlay изначально спроектирована для белой маркировки в формате SaaS: поддержка темизации для каждого тенанта, фильтрации каталога, OAuth2 и биллинга позволяет партнёрам запускать полностью брендированный сервис, а не просто поверхностно перекрашенный. При этом платформа полностью контейнеризована: каждый сервис, база данных, сборка, тестирование и сканирование работают в контейнерах, что делает развёртывание и проверку всей системы полностью воспроизводимыми. Как и остальные продукты семейства Helix, она живёт по принципу «антиблефа»: зелёный тест гарантирует реальное, пригодное для конечного пользователя поведение, а не просто успешное прохождение моков.

Зачем мы её создали

Коммерческие облачные игровые сервисы закрыты, централизованы и работают по модели аренды. HelixPlay создана для того, чтобы любой владелец машины с GPU мог запустить собственный стриминговый хост — открытый, саморазвёртываемый и поддерживающий белую маркировку, — вместо того чтобы зависеть от сторонних сервисов.

Почему это меняет правила игры

Здесь объединены три вещи, которые коммерческие сервисы держат по отдельности: самостоятельный хостинг на оборудовании под вашим контролем, единое ядро Go, управляющее тремя клиентскими стеками, благодаря чему функции появляются везде одновременно, и мультитенантность с возможностью белой маркировки. В результате партнёр может запустить полностью брендированный облачный игровой сервис на *своих* GPU — владея опытом, пользователями и экономикой — вместо того, чтобы перепродавать мощности чужого облака и жить в его ограничениях.

Что нового

- **Конвергенция трёх клиентских стеков** — Wails, Flutter и Angular работают на одном ядре Go (WASM в браузере), так что десктоп, мобильные устройства, телевизоры и веб используют единую реализацию вместо трёх расходящихся.
- **Самостоятельно развёртываемый SaaS с белой маркировкой** — встроенные на уровне каждого тенанта темизация, фильтрация каталога, OAuth2 и биллинг, благодаря чему платформа поставляется как готовый к брендированию продукт, а не как демо-версия.
- **Современный транспорт с низкой задержкой** — WebRTC (Pion), QUIC и кастомный UDP в сочетании с выбором аппаратного энкодера для каждой платформы (NVENC / QSV / AMF / VideoToolbox), оптимизированные под отзывчивость, а не удобство.
- **Архитектура из 46 независимых модулей** — чётко разделённые компоненты с контейнеризацией всего: каждый сервис, база данных, сборка, тестирование и сканирование работают в контейнерах.

Основные технические вызовы и их решения

- **Потоковая передача с низкой задержкой на разнородном оборудовании.** Каждая ОС и GPU по-разному реализуют захват и кодирование, а задержка не прощает ошибок. Решение — платформозависимый путь захвата и кодирования: DXGI / ScreenCaptureKit / PipeWire, передающие данные на NVENC / QSV / AMF / VideoToolbox через WebRTC / QUIC / UDP, чтобы каждая машина использовала свой самый быстрый путь к пикселям.
- **Единый продукт для десктопа, мобильных устройств, телевизоров и веба.** Решение — три клиентских стека

(Wails, Flutter, Angular), использующие одно ядро Go, компилируемое в WASM для браузера. Благодаря этому исправление или новая функция, написанные один раз, появляются на всех четырёх платформах сразу, а не портируются четырежды.

- **Мультитенантная работа с белой маркировкой.** Решение — встроенные в ядро бэкенда темизация, фильтрация каталога, OAuth2 и биллинг для каждого тенанта, так что изоляция и брендинг становятся базовыми примитивами платформы, а не кастомизацией под каждого клиента.

Технический стек

- **Go (корневая версия 1.26.2 / подмодули 1.25+)** — общее ядро бэкенда и хост-агент; один язык, компилируемый как в нативные бинарники, так и в WASM, что делает возможной архитектуру с единым ядром и несколькими клиентами.
- **Wails v2** — десктопный клиент, связывающий ядро Go с встроенным вебвью, чтобы десктопное приложение напрямую использовало логику ядра вместо её повторной реализации.
- **Flutter 3.29+** — клиент для мобильных устройств и телевизоров, обращающийся к ядру Go через FFI для нативного UI на телефонах и ТВ без второго бэкенда.
- **Angular 17+** — веб-клиент, запускающий то же самое ядро Go, скомпилированное в WASM, так что браузер становится полноценной платформой, а не урезанной версией.
- **WebRTC / Pion v4, QUIC / quic-go, кастомный UDP** — три транспортных протокола реального времени, позволяющие платформе выбирать путь с минимальной задержкой для каждой сети и клиента.
- **Аппаратные энкодеры (NVENC / QSV / AMF / VideoToolbox) и захват с платформы (DXGI / ScreenCaptureKit / PipeWire)** — ускоренный GPU путь захвата и кодирования, выбираемый под каждую платформу, чтобы кодирование никогда не становилось узким местом на CPU.
- **Контейнеры (Docker/Podman)** — каждый сервис, база данных, сборка, тестирование и сканирование работают в контейнерах, что делает всю систему воспроизводимой для развёртывания и проверки.
- **mDNS / rendezvous** — автоматическое обнаружение хоста без настройки, чтобы клиенты сами находили потоковый сервер в сети.

Заметки о статусе и честности

- **Статус: бета-версия.** Целевые показатели задержки в README (≤ 30 мс в локальной сети / ≤ 50 мс в глобальной сети по 99,9-му перцентилю), формулировки «консольный класс / класс PS4 Pro», а также количество ячеек в тестовой матрице заявлены самим проектом как собственные конструкторские цели, не подтверждённые независимым бенчмаркингом, и представлены именно в таком качестве.
- **Лицензия: не определена.** Файл LICENSE не обнаружен через GitHub API — НЕПРОВЕРЕНО / не объявлено.

Приоритетный уровень: Helix-основной.